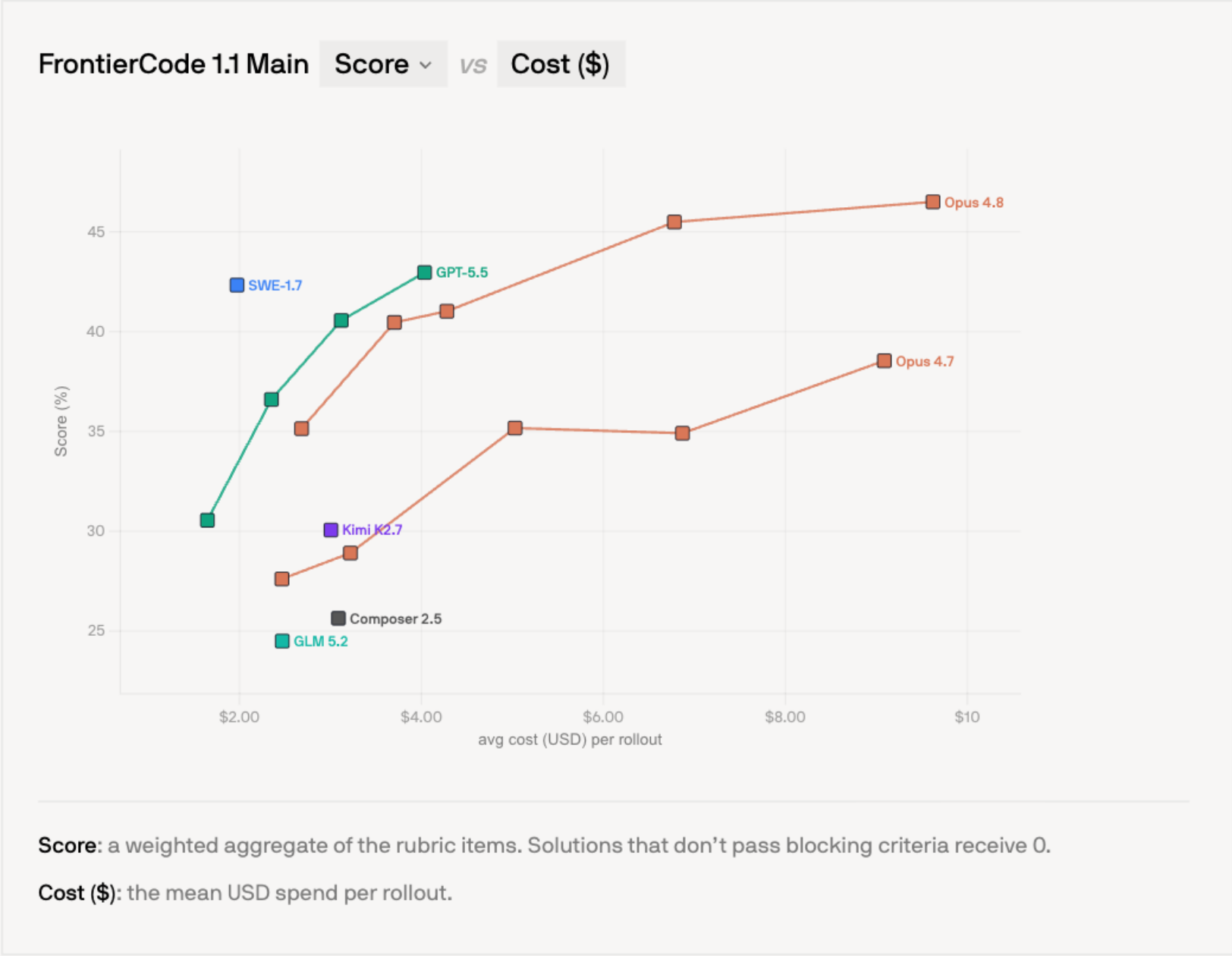
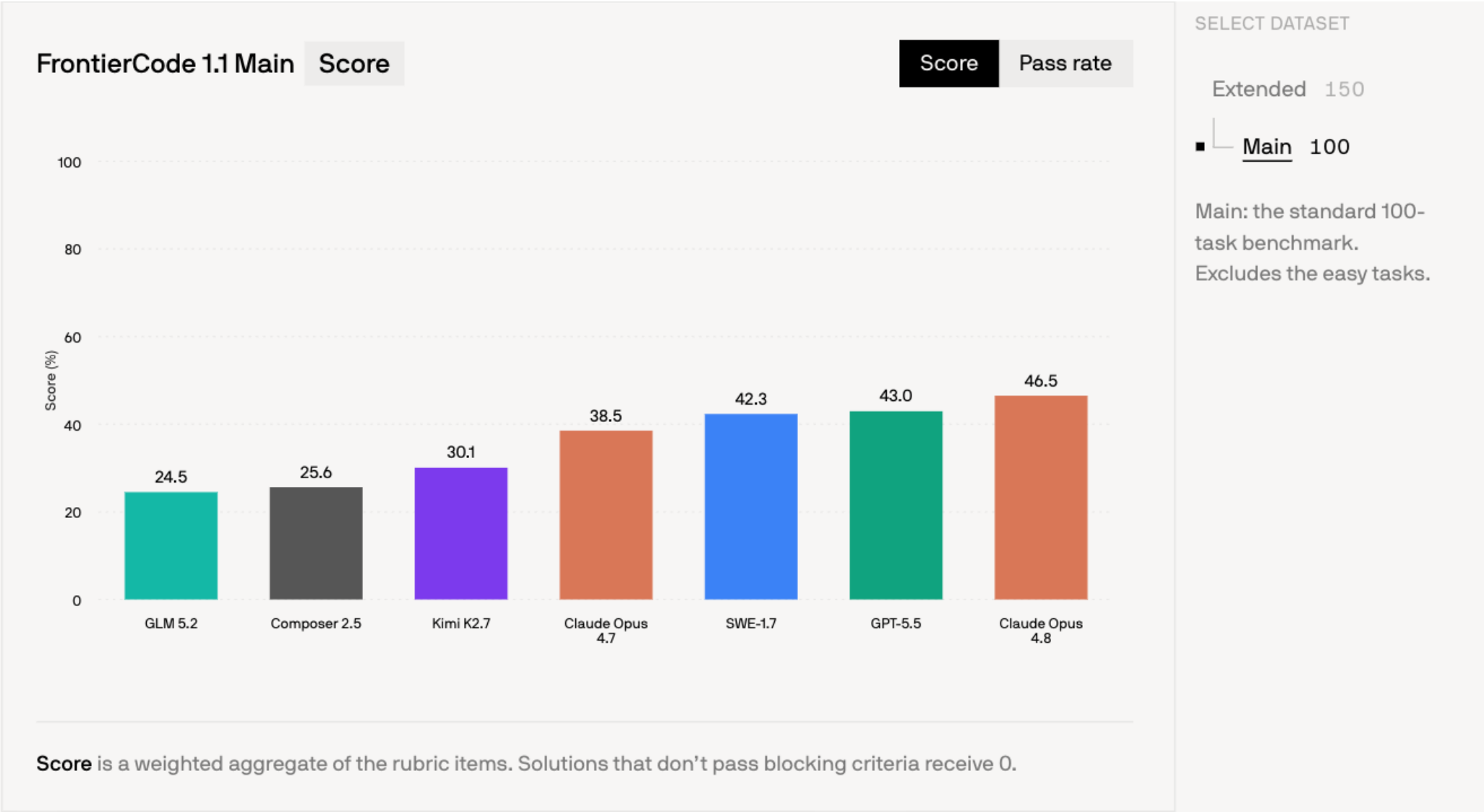






FrontierCode 1.1 Main

Score

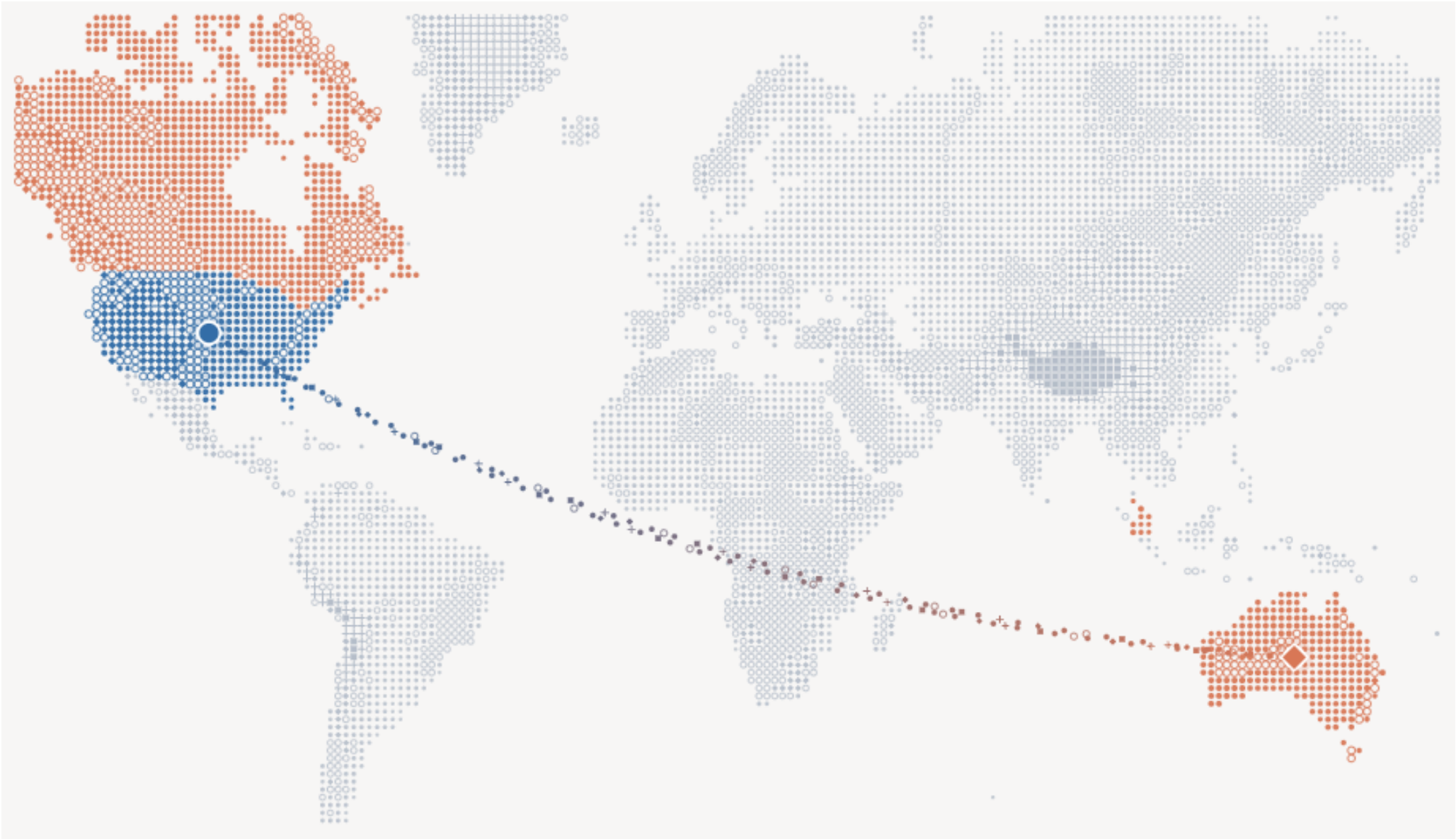
vs

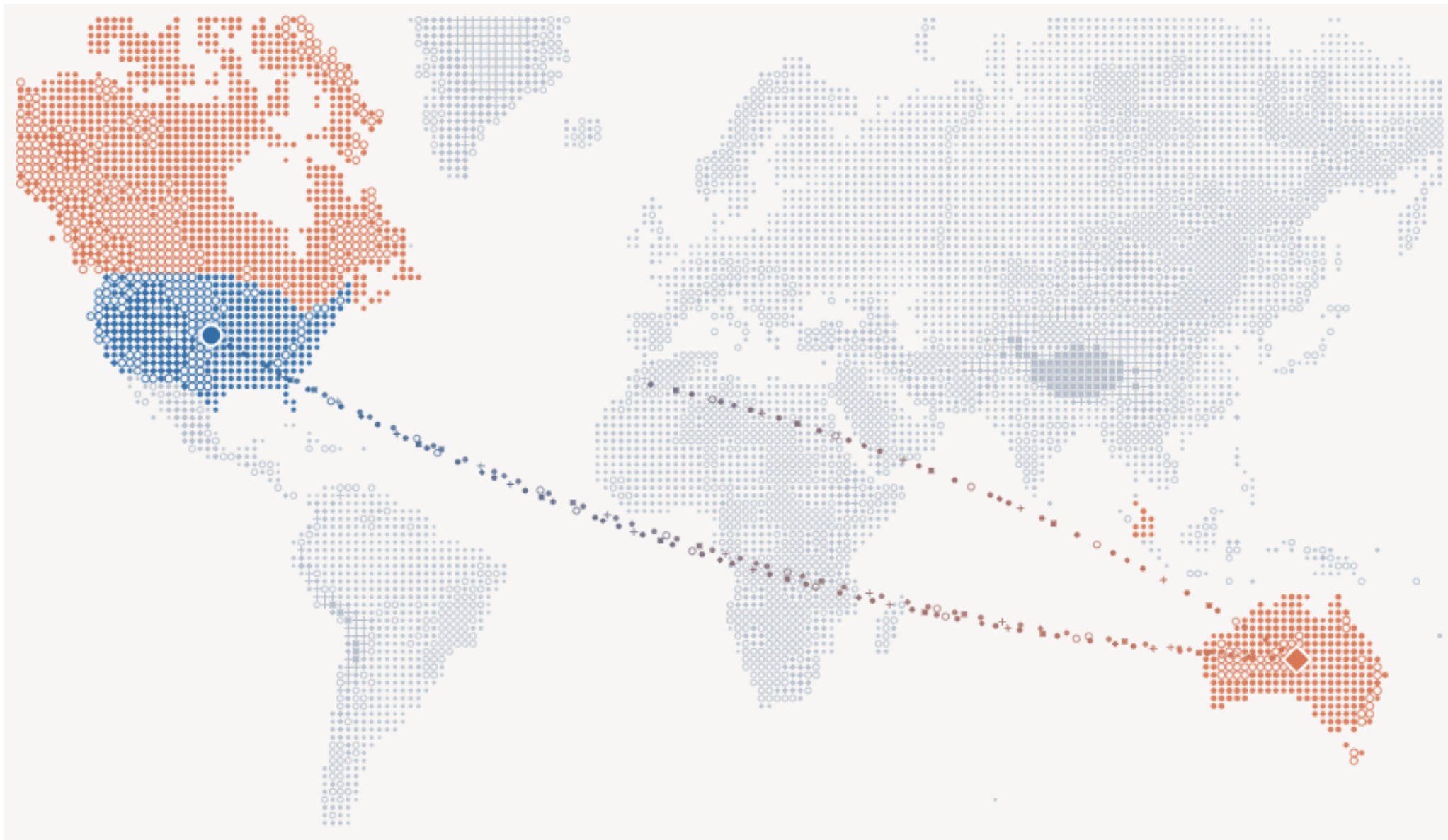
Cost (\$)

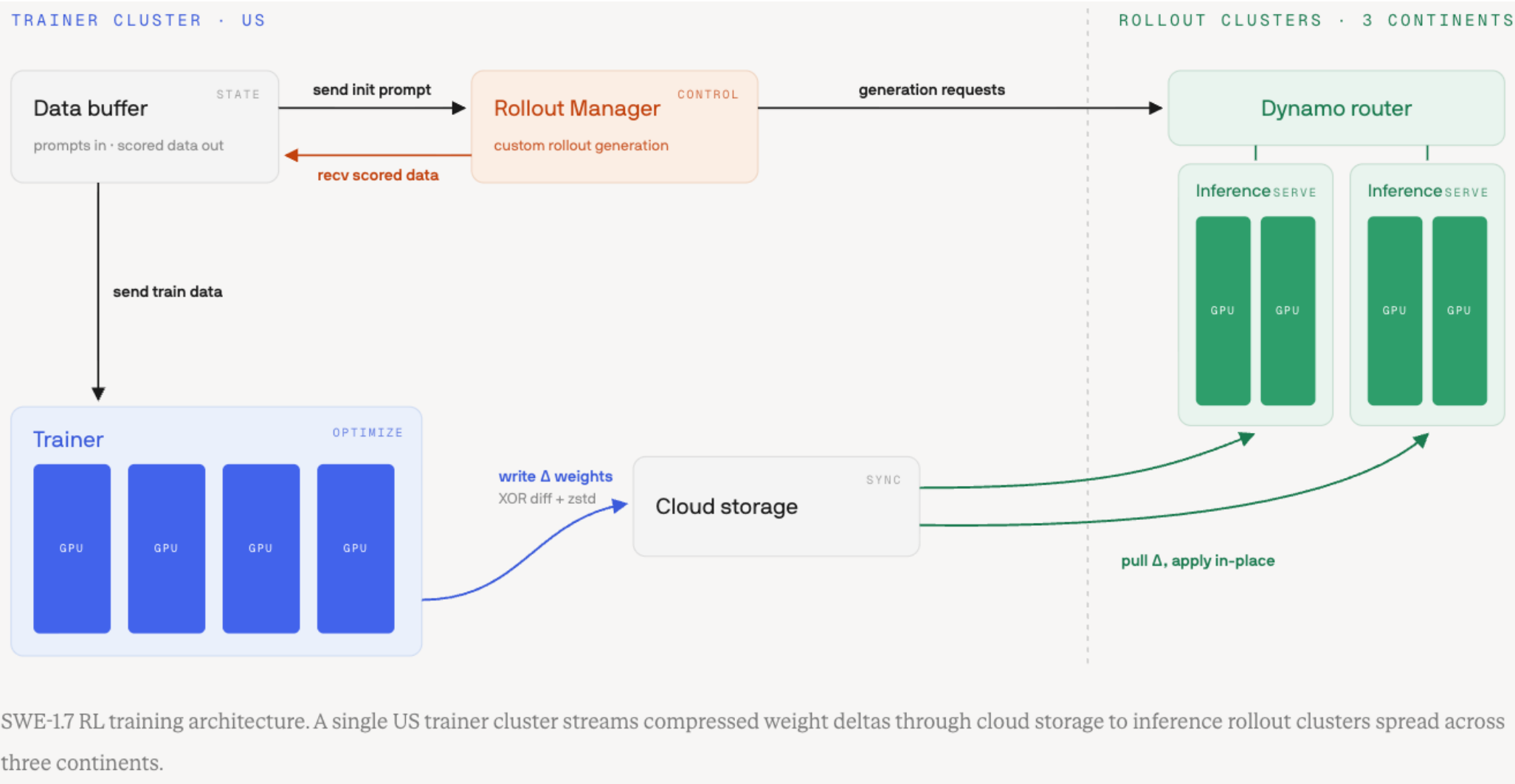
Model	avg cost (USD) per rollout	Score (%)
GLM 5.2	~\$1.5	~24.5
Composer 2.5	~\$2.5	~25.6
Kimi K2.7	~\$3.2	~30.1
SWE-1.7	~\$2.2	~42.3
GPT-5.5	~\$4.0	~43.0
Opus 4.7	~\$9.0	~38.5
Opus 4.8	~\$9.5	~46.5

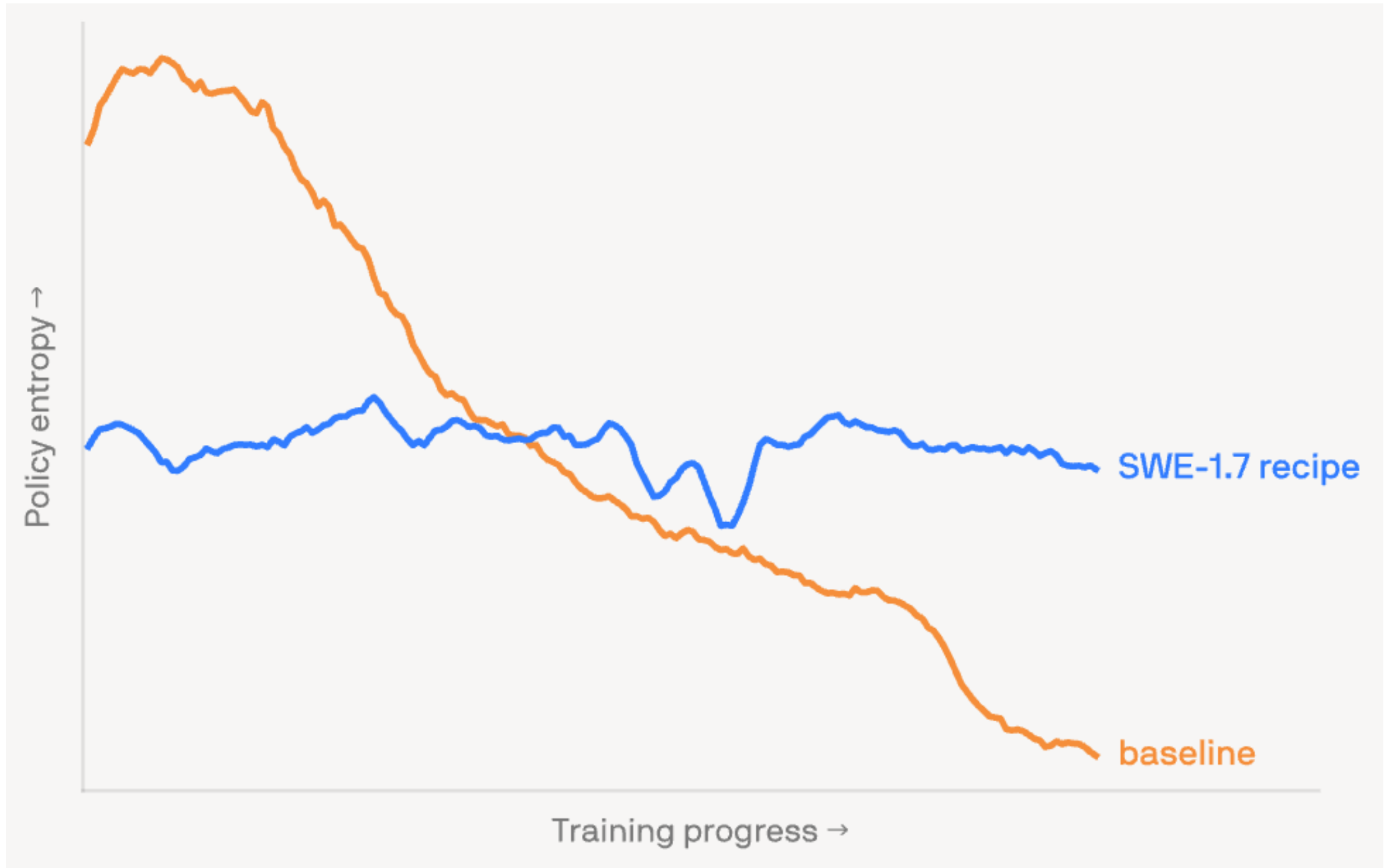
Score: a weighted aggregate of the rubric items. Solutions that don't pass blocking criteria receive 0.

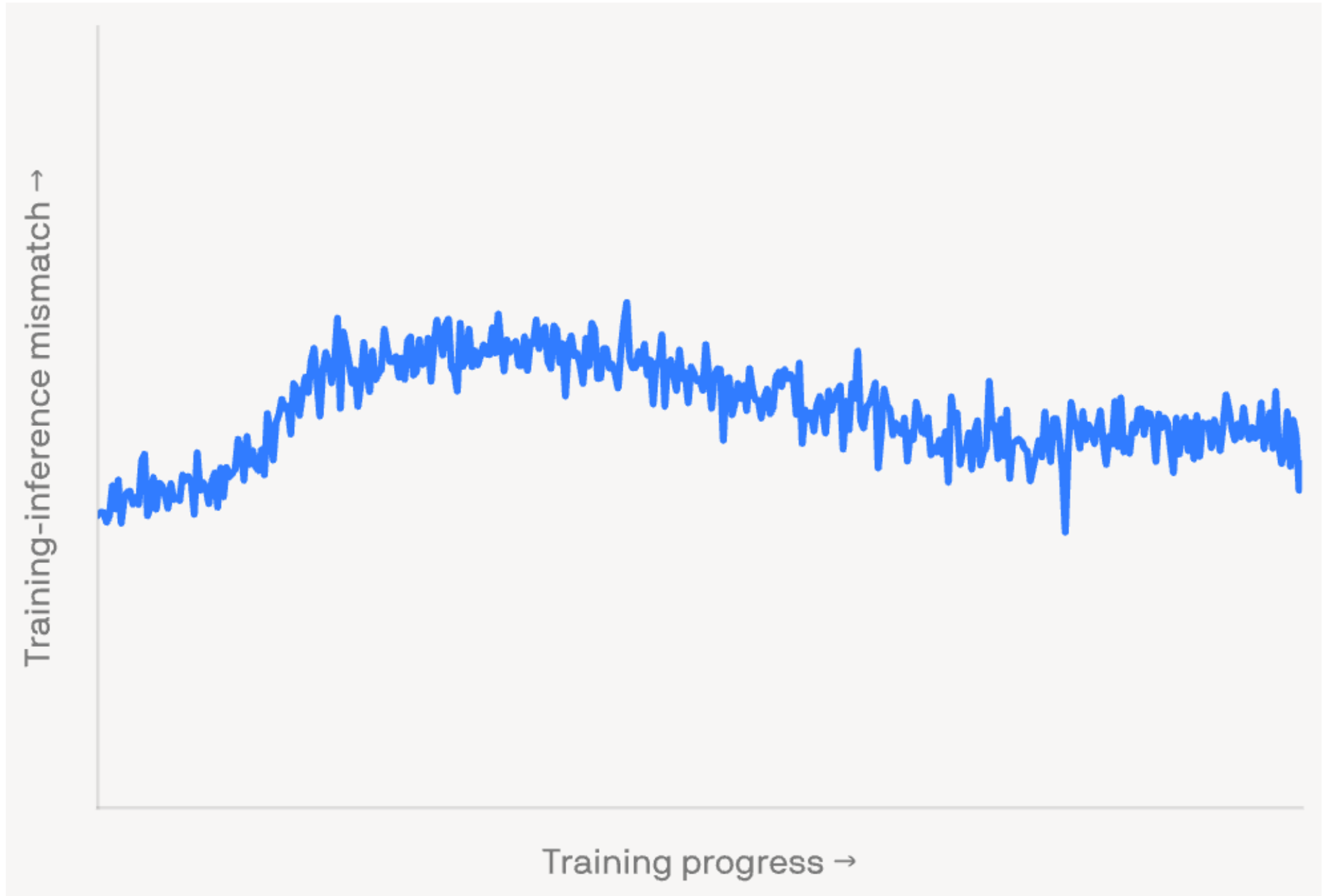
Cost (\$): the mean USD spend per rollout.

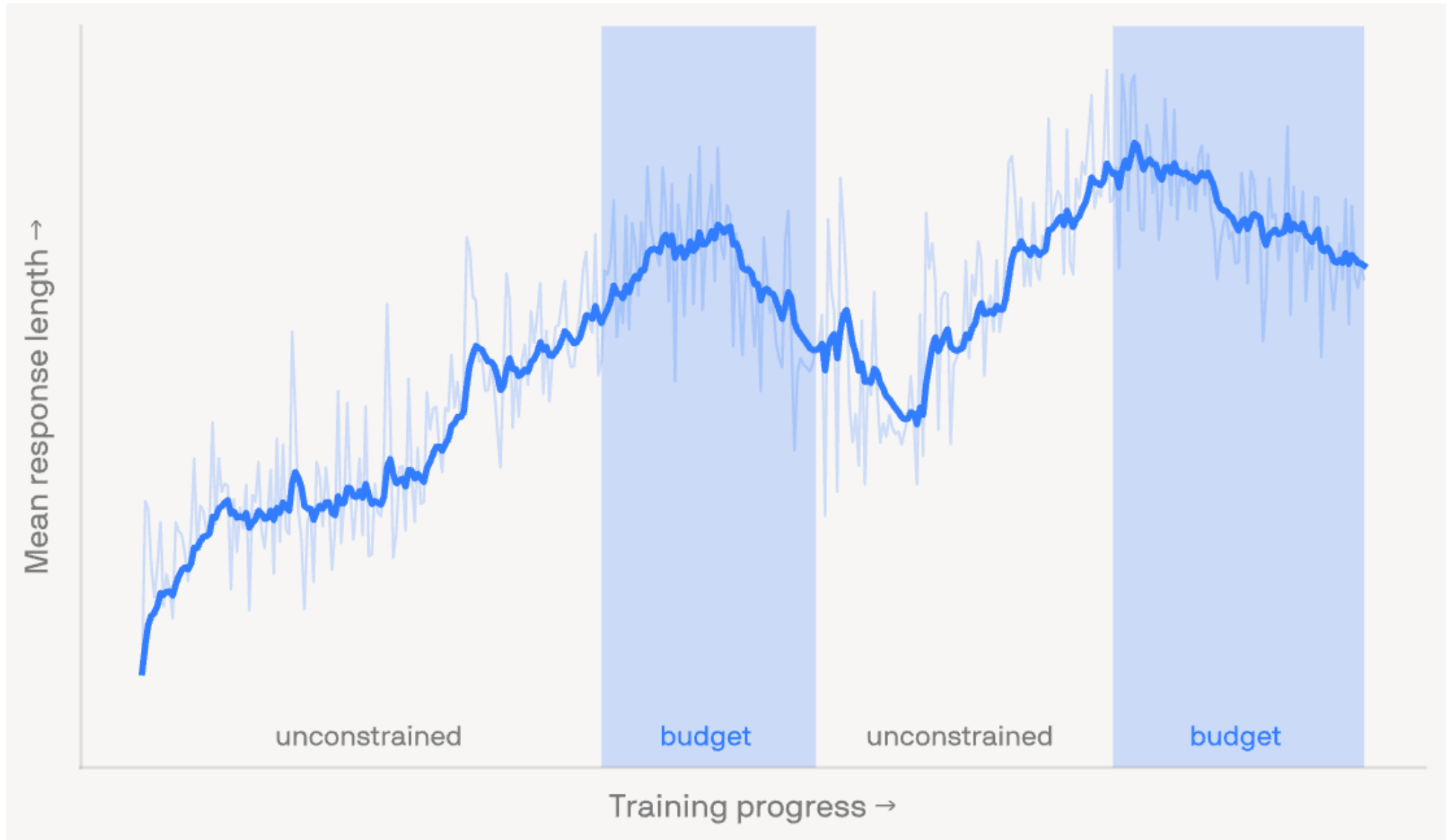


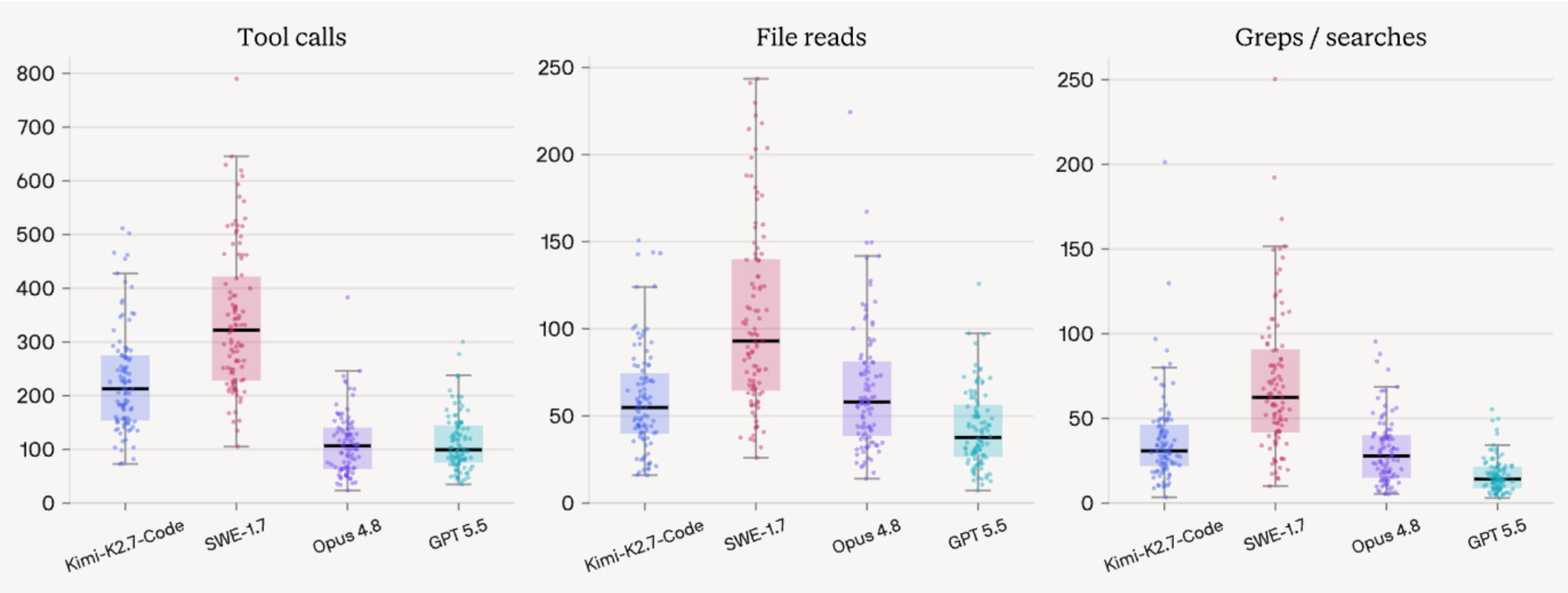


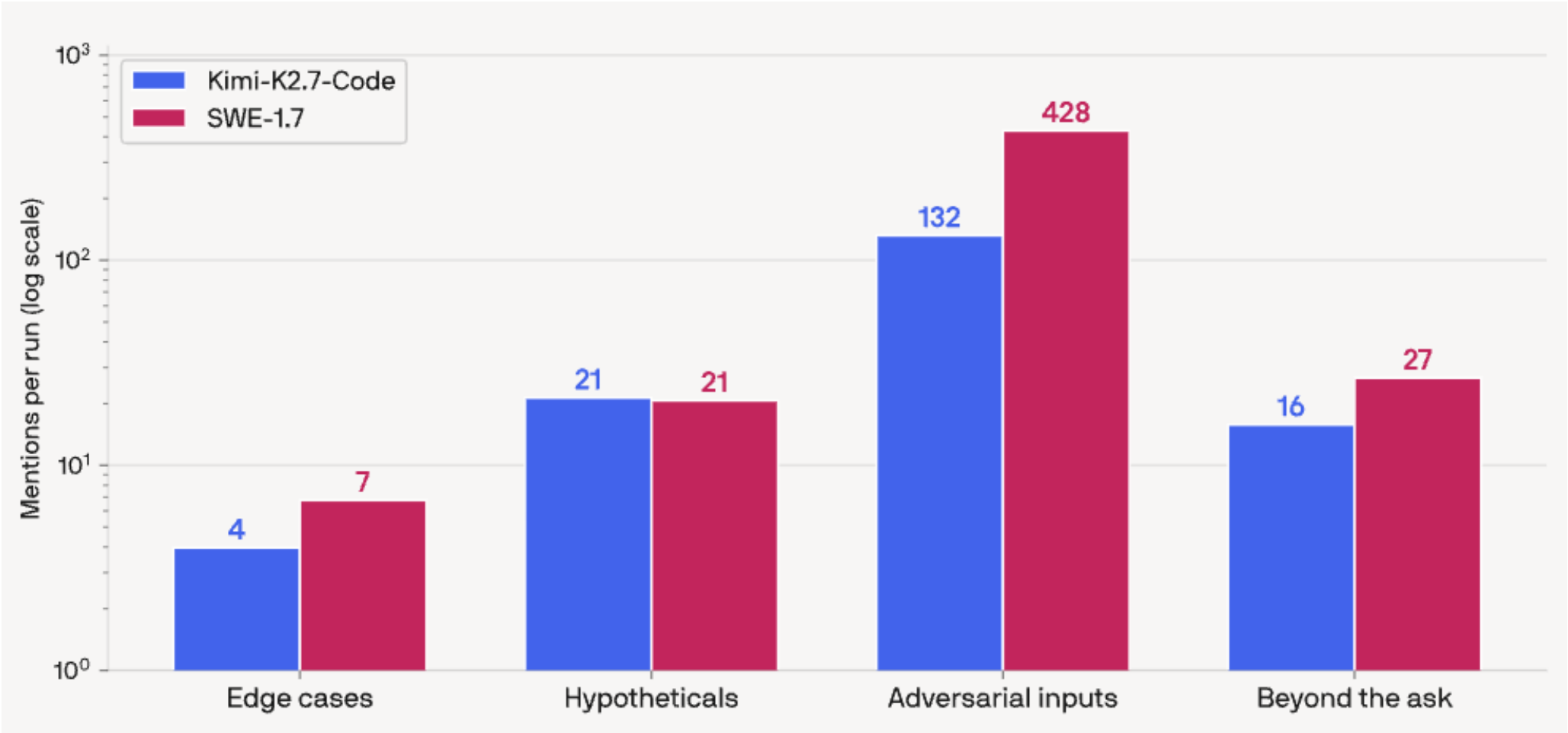












Condensed chain-of-thought		Hide
SWE-Bench Pro task: protonmail/webclients-b387b241 ansible/ansible-1a4644ff future-architect/vuls-5af1a227		
Kimi-K2.7-Code		SWE-1.7
THINKING BLOCK #1 Let me start by exploring the repository structure to understand what we're working with. TOOL CALL todo_write TOOL CALL find_file_by_name TOOL CALL grep THINKING BLOCK #2 Let me read the main psrp.py connection plugin and the test file to understand the current behavior and expected changes. TOOL CALL read x2 THINKING BLOCK #3 Now let me read the rest of psrp.py, especially the _build_kwargs method.		THINKING BLOCK #1 We need modify repository in /app. Need explore code relevant psrp connection plugin. User asks implement changes. We need find psrp connection plugin file. Also need create reproduce script, execute, confirm error, edit, rerun. Must not modify tests. Need thorough. Use tools. Need be careful with mode? It is normal. We can explore. We need locate ansible/lib/ansible/plugins/connection/psrp.py perhaps. Let's find. TOOL CALL find_file_by_name TOOL CALL grep THINKING BLOCK #2 Need read psrp.py fully. Also test file to understand expected behavior. And plugins/__init__.py maybe. Need understand _psrp_conn_kwargs, allow_extras, AUTH_KWARGS. Let's read. TOOL CALL read x2

Condensed chain-of-thought		Hide
SWE-Bench Pro task: protonmail/webclients-b387b241 ansible/ansible-1a4644ff future-architect/vuls-5af1a227		
Kimi-K2.7-Code		SWE-1.7
THINKING BLOCK #1 Let me start by exploring the repository structure to understand the codebase. TOOL CALL todo_write THINKING BLOCK #2 Let me explore the repo structure and find relevant files. TOOL CALL find_file_by_name TOOL CALL grep x2 THINKING BLOCK #3 Let me read the relevant files. TOOL CALL read x3 THINKING BLOCK #4		THINKING BLOCK #1 We need modify repo. Need understand structure. Let's start exploring /app. Also use todo list. We need read relevant files: oval/redhat.go, scanner/redhatbase.go, constants. Need create reproduce script. Need minimal non-test changes. Need ensure correct kernel matching. Let's investigate. TOOL CALL todo_write THINKING BLOCK #2 Let's list files and read. TOOL CALL find_file_by_name x3 THINKING BLOCK #3 Read relevant files. TOOL CALL read x2 THINKING BLOCK #4

Condensed chain-of-thought		Hide
SWE-Bench Pro task: protonmail/webclients-b387b241 ansible/ansible-1a4644ff future-architect/vuls-5af1a227		
Kimi-K2.7-Code		SWE-1.7
THINKING BLOCK #1 Let me start by exploring the repository structure to understand the codebase and find the relevant files mentioned in the PR description. TOOL CALL todo_write		THINKING BLOCK #1 We need follow instructions. Need inspect repo, find relevant files, create reproduce script, implement changes, rerun. First explore. Let's use tools. Need maybe todo list because multi-step. We'll create todos. Need search for files useMyCountry.tsx, PhoneInput.tsx, etc. Use find_file_by_name or grep. TOOL CALL todo_write
THINKING BLOCK #2 Let me explore the repository structure and find the relevant files. TOOL CALL find_file_by_name x9		THINKING BLOCK #2 Now find files. Use find_file_by_name patterns. TOOL CALL find_file_by_name x9
THINKING BLOCK #3 Good, I found all the files. Let me read them in parallel to understand the current implementation. TOOL CALL read x5		THINKING BLOCK #3 We have multiple CustomStep.tsx files. PR lists "CustomStep.tsx" maybe generic? Need determine which ones use useMyCountry. Let's grep for useMyCountry usage. TOOL CALL grep
THINKING BLOCK #4		

Example trajectories		Hide
FrontierCode 1.1 Main task: fizzy-implement-authorization matplotlib-bug-stackplot uv-frozen-uv-lock-check-fails		
Deriving the hidden contract: The task leaves key scenarios unspecified, and the hidden tests check them. SWE-1.7 derives the missing requirements by enumerating every possible requester, tracing the hostile cross-account case through the app's real session code, and proving the vulnerability with a failing test before fixing anything. Kimi answers the same questions from intuition and writes tests that encode its own assumptions, which ultimately fail.		
Kimi-K2.7-Code	SWE-1.7	
WHAT'S HAPPENING <i>Kimi explores the app thoroughly and reaches the key design question: every account lives under its own URL prefix, so what if a file request arrives without one — with no account context at all? It is exactly the right question. But instead of tracing the code or firing a test request, Kimi reassures itself that the app's URLs will always include the account — and builds the entire design on that assumption: every request must pass account-membership checks before file permissions are even looked at.</i> THINKING BLOCK But what if it's just <code>/rails/active_storage/...</code> without account prefix? Then <code>Current.account</code> is <code>nil</code>.[...]Actually, <code>ActiveStorage</code> URLs generated in the app will include the <code>script_name</code>. So they will be <code>'/{account_id}/rails/active_storage/blobs/...'</code>. This means <code>Current.account</code> will be set. WHAT'S HAPPENING <i>Next, the two cases the task never specifies. Kimi settles</i>	WHAT'S HAPPENING <i>SWE spots the trap immediately: you can't just reuse the app's login check on the file endpoints, because it would bounce every anonymous visitor — including people viewing legitimately public boards. So it designs its own order of checks: public files first, then permissions for logged-in users, a login redirect for anonymous ones, and "not found" only for logged-in users without access. That ordering even gives the right answer for the orphaned-file case it never explicitly considers.</i> THINKING BLOCK But for public access, we cannot simply include <code>Authentication</code> and run <code>require_authentication</code> because it would redirect all unauth requests. We need allow unauth for avatars and public blobs. So in <code>ActiveStorage</code> controllers, we need a custom flow:[...]4. If not authenticated and not public, redirect to login. 5. If authenticated and not allowed, <code>head :not_found</code> (or <code>forbidden</code>). WHAT'S HAPPENING	

Example trajectories		Hide
FrontierCode 1.1 Main task: fizzy-implement-authorizationmatplotlib-bug-stackplotuv-frozen-uv-lock-check-fails		
Empirical Probing: SWE-1.7 treats ambiguity as something it resolves by exploring the codebase or probing it directly. On this task, SWE-1.7 writes ~30 tiny Python probes before its first edit to learn how matplotlib actually interprets each ambiguous input, while Kimi answers the same questions in its head and runs zero code before editing. The hidden tests probe exactly those ambiguous cases, so SWE's fix encodes measured answers while Kimi's encodes unchecked guesses.		
Kimi-K2.7-Code	SWE-1.7	
WHAT'S HAPPENING <i>Kimi finds the bug on its first read of the code. It even notices the task asks for more than the crash fix: the other style options (edgecolor, linewidth, linestyle) need the same treatment (ev 5-37).</i> THINKING BLOCK I can see the problem. The `stackplot` function passes `facecolor=next(colors)` to every `fill_between` call. If the user also passes `facecolor` or `facecolors` in `kwargs`, it will conflict THINKING BLOCK We also need to consider `edgecolor`, `linewidth`, `linestyle` consistency. WHAT'S HAPPENING <i>Then it talks itself out of that requirement. Whether the current behavior is "fine" is a question one quick script could answer — Kimi answers it in its head instead, and</i>	WHAT'S HAPPENING <i>SWE reads the same code and finds the same bug. The difference starts with what it does about the open questions (ev 5-22).</i> THINKING BLOCK The bug: stackplot passes facecolor=next(colors) and **kwargs to fill_between. If user supplies `facecolor='red'` in kwargs, duplicate argument. WHAT'S HAPPENING <i>Before designing anything, SWE writes a small throwaway script that feeds all ten style options to the broken stackplot and prints what each one does. Now it knows the full damage: which options crash, which quietly work (ev 191-192).</i> TOOL CALL for kw in [{'facecolor':'red'}, {'facecolors':'red'}, {'facecolor': ['red', 'blue']}, {'facecolors': ['red', 'blue']}]	

Example trajectories		Hide
FrontierCode 1.1 Main task: fizzy-implement-authorization matplotlib-bug-stackplot uv-frozen-uv-lock-check-fails		
Generalize the fix to the evidence, not the instruction: This task tests whether a model fixes the whole class of bugs or just the one instance the instruction mentions — the flaw lives in shared machinery, and hidden tests check the wider surface area. Both models detect that the bug is broader, but Kimi repeatedly reasons itself back to the narrower instruction, while SWE-1.7 frames the job as general from the start and ships the fix for the whole class.		
Kimi-K2.7-Code	SWE-1.7	
WHAT'S HAPPENING <i>Kimi nails the diagnosis and sees the bug is bigger than the ticket. It quickly understands why 'UV_FROZEN=0' misbehaves (the parser checks whether a flag is present, never its value), and notices, unprompted, that other commands like 'uv sync' hit the identical flaw. Every piece of evidence points at a general fix.</i> THINKING BLOCK But clap's conflict system triggers on the presence of the arg, not its value. So this also conflicts, which is the bug. THINKING BLOCK If I keep 'conflicts_with', then 'UV_FROZEN=0 uv sync --no-sync' would still conflict (because frozen from env is considered present). Is that desired? The user didn't mention that case, but it's the same bug. WHAT'S HAPPENING <i>Kimi retreats to the instruction's narrower scope. While it</i>	WHAT'S HAPPENING <i>SWE reproduces first, and generalizes on the spot. It builds the real binary, triggers the bug, and its very first analysis reframes the job: not one command's quirk, but a general rule — a false env var must never cause a conflict.</i> TOOL CALL UV_FROZEN=0 /home/ubuntu/repos/uv/target/debug/uv lock --check THINKING BLOCK Both 0 and 1 conflict. The issue specifically says 'UV_FROZEN=0' should not. [...] Need fix general env bool false causing conflicts. WHAT'S HAPPENING <i>It rejects the shortcut and builds the fix for the whole class. SWE considers hardcoding the handful of env vars it has seen so far — and turns the idea down. Its fix instead walks every argument of every uv subcommand automatically</i>	